

< 200 CONCRETE OPEN PROBLEMS IN MECHANISTIC INTERPRETABILITY >

200 COP in MI: Exploring Polysemanticity and Superposition

by **Neel Nanda** 3rd Jan 2023

Important Note: Since writing this, there's been a lot of exciting work on understanding superposition via [training sparse autoencoders](#) to take features out of superposition. I recommend reading up on that work, since it substantially changes the landscape of what problems matter here.

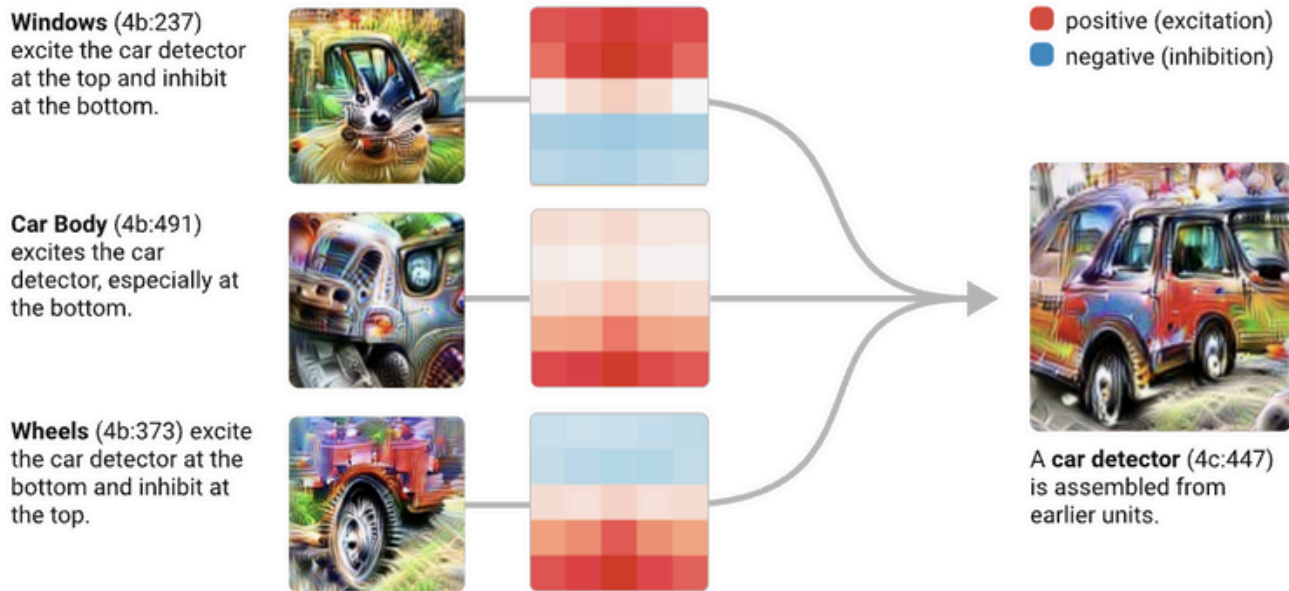
This is the fifth post in a sequence called 200 Concrete Open Problems in Mechanistic Interpretability. [Start here](#)[°], then read in any order. If you want to learn the basics before you think about open problems, check out [my post on getting started](#). Look up jargon in [my Mechanistic Interpretability Explainer](#)

Motivating papers: [Toy Models of Superposition](#), [Softmax Linear Units](#)

Background

If you're familiar with polysemanticity and superposition, skip to Motivation or Problems.

Neural networks are very high dimensional objects, in both their parameters and their activations. One of the key challenges in Mechanistic Interpretability is to somehow resolve the curse of dimensionality, and to break them down into lower dimensional objects that can be understood (semi-)independently.



Our current best understanding of models is that, internally, they compute **features**: specific properties of the input, like “this token is a verb” or “this is a number that describes a group of people” or “this part of the image represents a car wheel”. That early in the model there are simpler features, are later used to compute more complex features by being connected up in a **circuit** (example shown above ([source](#))). Further, our guess is that features correspond to **directions in activation space**. That is, for any feature that the model represents, there is some vector corresponding to it. And if we dot product the model’s activations with that vector, we get out a number representing whether that feature is present. (these are known as **decomposable, linear representations**)

This is an extremely useful thing to be true about a model! An even more helpful thing to be true would be if *neurons* correspond to features (ie the output of an **activation function** like ReLU). Naively, this is natural for the model to do, because a non-linearity like ReLU acts element-wise - each neuron’s activation is computed independently (this is an example of a **privileged basis**). Concretely, if a neuron can represent feature A or feature B, then that neuron will fire differently for feature A and NOT feature B, vs feature A and feature B, meaning that the presence of B **interferes** with the ability to compute A. But if each feature is its own neuron we’re fine!

If features correspond to neurons, we’re playing interpretability on easy mode - we can focus on just figuring out which feature corresponds to each neuron. In theory we could even show that a feature is *not* present by verifying that it’s not present in each neuron! However, reality is not as nice as this convenient story. A countervailing force is the phenomena of **superposition**. Superposition is when a network represents more features than it has

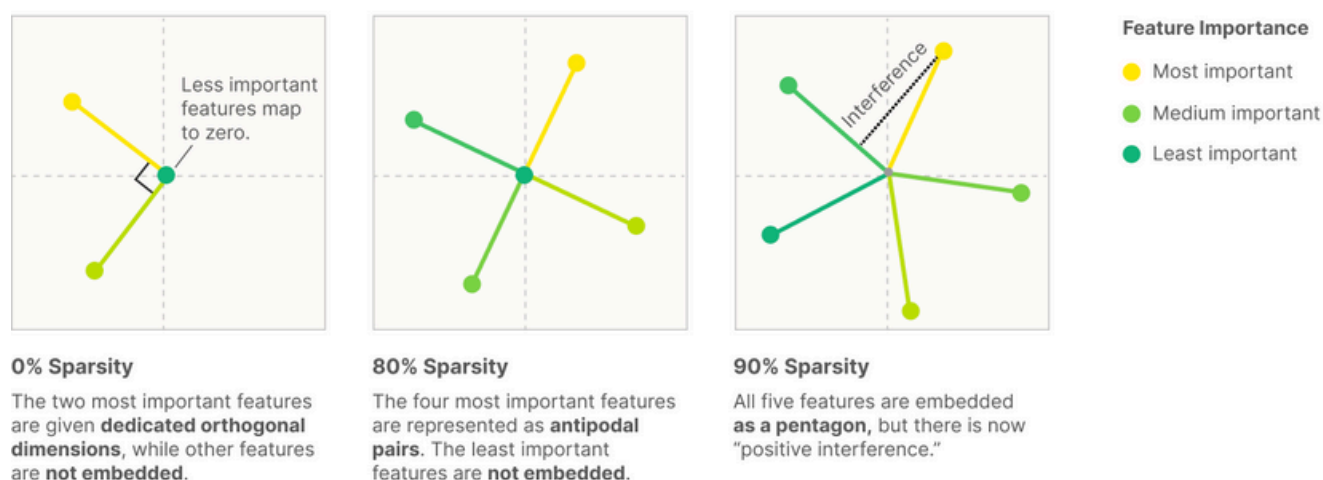
dimensions, and squashes them all into a lower dimensional space. You can think of superposition as the model **simulating a larger model**.

Anthropic's [Toy Models of Superposition](#) paper is a great exploration of this. They build a toy model that learns to use superposition (notably different from [a toy language model](#)!). The model starts with a bunch of independently varying features, needs to compress these to a low dimensional space, and then is trained to recover each feature from the compressed mess. And it turns out that it does learn to use superposition!

Specifically, it makes sense to use superposition for sufficiently rare (**sparse**) features, if we give it non-linearities to clean up interference. Further, the use of superposition can be modelled as a trade-off between the costs of interference, and the benefits of representing more features. And digging further into their toy models, they find all kinds of fascinating motifs regarding exactly *how* superposition occurs, notably that the features are sometimes **compressed in geometric configurations**, eg 5 features being compressed into two dimensions as the vertices of a pentagon, as shown below.

As Sparsity Increases, Models Use "Superposition" To Represent More Features Than Dimensions

Increasing Feature Sparsity →



Motivation

Zooming out, what does this mean for what research actually needs to be done? To me, when I imagine what real progress here might look like, I picture the following:

1. **Crisp conceptual frameworks:** I still feel pretty confused about what is even going on with superposition! How much does it occur? The Toy Models paper significantly clarified my intuitions, but it's far from complete. I expect progress here to mostly look

like identifying the aspects of transformers and superposition that we're still confused about, building toy models to model those, and seeing what insights can be learned

2. **Empirical data from real models:** It's all well and good to have beautiful toy models and conceptual frameworks, but it's completely useless if we aren't learning anything about real models! I would /love to have some well-studied cases of superposition and polysemanticity in real models, and to know whether any of the toy model's predictions transfer.
 - a. Can we find *any* truly monosemantic neurons? Can we find a pentagon of features in a real residual stream? Can we reverse engineer a feature represented by several neurons?
3. **Dealing with superposition in practice:** Understanding superposition is only useful in that it allows us to better understand networks, so we need to know how to deal with it in practice! Can we identify all directions that correspond to features? Can we detect whether a feature is at all neuron-aligned, or just an arbitrary direction in space?

The direction I'm most excited about is a combination of 1 and 2, to form a rich feedback loop between toy models and real models - toy models generate hypotheses to test, and exploring real models generates confusions to study in toy models.

Resources

- The [Toy Models of Superposition](#) paper. This is a fascinating and well-written paper, and **I recommend reading it before working on a problem in this area!** There's a ton more insights in there that I didn't describe here.
 - A [video walkthrough](#) I made for it.
 - [Their accompanying Colab](#)
 - Their [motivation section](#) heavily inspired the introduction to this post, and is a great distillation of why this matters.
- The sections of my MI explainer on [superposition](#) and on [the toy models of superposition](#) paper
- The [Softmax Linear Units \(SoLU\)](#) paper, especially the section laying out [background on how to think about superposition](#).
 - See [my explainer section on SoLU](#) for another distillation
- My (under construction!) [neuroscope](#) website that shows the max activating dataset examples for each neuron in some language models.

- When studying evidence in real models, I expect that my toy language models will be easiest to study (check out [the resources for that post](#)^o, and [load them in TransformerLens](#)). There are 12 models, from 1 to 4 layers, and one of each attention-only, GELU activation MLPs and SoLU activation MLPs
 - Note - toy language models = normal language models but scaled down (only 1-4 layers), or without MLP layers. But toy models = a specific set up designed to simulate something interesting in a larger model. In some sense, toy language models are just a special kind of toy model, but these terms are similar and can be confusing!
- I also have [some larger SoLU models](#) (available in TransformerLens), up to GPT-2 Medium size

Tips

- A common feeling in people new to the field is that toy model work is easy, and working with real transformers is hard. If anything, I would argue the opposite. The core difficulty of working with toy models is not analysing the model per se, but rather finding the *right* model to analyse. It's a delicate balance between being a true simulation of what we care about in a real model, and simple enough to be tractable to analyse, and it's very easy to go too far in either direction.
 - I have seen several toy model projects fail, where even though the toy model itself was interesting, they'd failed to capture some key part of the underlying problem.
 - For example, when I first tried to explore the toy models of superposition setup, I put a ReLU on the hidden dimension and *not* on the output. This looked very interesting at first, but in hindsight was totally wrong-headed! (Take a moment to try to figure out why before you read on!)
 - The model already *has* all the features, and it wants to use the bottleneck to *compress* these features. ReLUs are for computing new features and create significant interference between dimensions, so it's actively unhelpful on the bottleneck. But they're key at the *end*, because they're used for the "computation" of cleaning up the noise of interference with other features.
 - In practice, the model learned a large positive hidden bias so the hidden ReLUs always fired and just became a linear layer! And a large negative bias on the output to cancel that out.

- I recommend first doing projects that involve studying real language models and getting an intuition for how they work and what's hard about reverse engineering them, and using this as a bedrock to build and study a toy model.
 - The easiest way to do this is to have a mentor who can help you find a good toy model, and correct you when you go wrong. But finding a good mentor is hard!
- The right mindset for a toy model project is to take the process of setting up the toy model *really* seriously.
 - Find something about a transformer that you're confused about, and try to distill it down to a toy model.
 - Then try to red-team it, and think through ways it's disanalogous real models, and note down all of the assumptions you're making. (Easier to do with a friend! Outside perspectives are great)
 - Then try to actually analyse the toy model, regularly keeping in mind the confusion about real models that you're trying to understand, and checking in on whether you've lost track.
 - As you go deeper, you'll likely see ways the toy model could be more analogous, and can tweak the setup to be more true to the underlying confusion
- Useful clarification: Transformers actually have [two conceptually different kinds of superposition](#) internally, what I call **linear bottleneck superposition** and **neuron superposition**.
 - Bottleneck superposition is about **compression**. It occurs when there's a linear map from a high dimensional space to a low dimensional space, and then a linear map back to a high dimensional space *without* a non-linearity in the middle. Intuitively, the model already *has* the features in the high dimensional space, but wants to map them to the low dimensional space in a way such that they can be recovered later for further computation. But it's not trying to compute *new* features.
 - The residual stream, and queries, keys and values in attention heads are the main places this happen.
 - This is the main kind studied in Toy Models
 - Intuitively, this *must* be happening - in GPT-2 Small there is a vocabulary of 50,000 possible input tokens, which are embedded to a residual stream of 768 dimensions, yet GPT-2 Small can still tell the difference between the tokens!

- Neuron superposition is about **computation**. It occurs when there's more features than neurons, immediately after a non-linear activation function. Importantly, this means that the model has somehow *computed* more features than it had neurons - because the model needed to use a non-linearity, these features were not previously represented as directions in space.
 - This happens in transformer MLP layers.
 - This was studied in [section 8 of Toy Models](#), and they found the [fascinating asymmetric superposition motif](#), but it was not a focus of the paper.
 - It's not obvious to me that this is even in the model's interests (non-linearities make the interference between different features way higher!) but it seems like it does
- In my opinion, neuron superposition seems inherent to understanding what features the model knows and reverse engineering how they're computed, and thus more important to understand. And I am way more confused about it, so I'd be particularly excited to see more work here!
- Useful clarification 2: There are [two conceptually different kinds of interference](#) in a model, what I call **alternating interference** and **simultaneous interference**. Let's consider the different cases when one direction represents both feature A and feature B.
 - Alternating interference occurs when A is present and B is *not* present, and the model needs to figure out that despite there being some information along the direction, B is *not* present, while still detecting how much A is present. In toy models, this mostly seem to have been done by using ReLU to round off small activations to zero.
 - Simultaneous interference occurs when A is present *and* B is present, and the model needs to figure out that both are present (and how much!)
 - Their toy models mostly learn to deal with alternating interference and just break with simultaneous interference. If two features are independent and occur with probability p , then alternating interference occurs with probability $\sim 2p$ and simultaneous with p^2 . For small p , simultaneous interference just doesn't matter!

Problems

[This spreadsheet](#) lists each problem in the sequence. You can write down your contact details if you're working on any of them and want collaborators, see any existing work or

reach out to other people on there! (thanks to Jay Bailey for making it)

Notation: **ReLU output model** is the main model in the Toy Models of Superposition paper which compresses features in a linear bottleneck, **absolute value model** is the model studied with a ReLU hidden layer and output layer, and which uses neuron superposition.

- Confusions about models that I want to see studied in a toy model:
 - **A* 4.1** - Does dropout create a privileged basis? Put dropout on the hidden layer of the ReLU output model and study how this changes the results. Do the geometric configurations happen as before? And are the feature directions noticeably more (or less!) aligned with the hidden dimension basis?
 - **B-C* 4.2** - Replicate their absolute value model and try to study some of the variants of the ReLU output models in this context. Try out uniform vs non-uniform importance, correlated vs anti-correlated features, etc. Can you find any more motifs?
 - **B* 4.3** - Explore neuron superposition by training their absolute value model on a more complex function like $x \rightarrow x^2$. This should need multiple neurons per function to do well
 - **B* 4.4** - What happens to their ReLU output model when there's non-uniform sparsity? Eg one class of less sparse features, and another class of very sparse features.
 - Explore neuron superposition by training their absolute value model on functions of multiple variables:
 - **A* 4.5** - Make the inputs binary (0 or 1), and look at the AND or OR of pairs of elements
 - **B* 4.6** - Keep the inputs as uniform reals in $[0, 1]$ and look at $\max(x, y)$
 - Adapt their ReLU output model to have a different range of feature values, and see how this affects things. Currently the features are uniform $[0, 1]$ if on (and 0 if off):
 - **A* 4.7** - Make the features 1 (ie exactly two possible values)
 - **B* 4.8** - Make the features discrete, eg 1, 2 or 3
 - **B* 4.9** - Make the features uniform $[0.5, 1]$
 - **A-B* 4.10** - What happens if you replace ReLUs with GELUs in their toy models? (either for the ReLU output model, or the absolute value model). Does it just act like a smoother ReLU?

- **C* 4.11** - Can you find a toy model where GELU acts significantly differently from ReLU? A common intuition is that GELU is mostly a smoother ReLU, but close to the origin GELU can act more like a quadratic. Does this ever matter?
- **C* 4.12** - Build a toy model of a classification problem, where the loss function is cross-entropy loss (not mean squared error loss!)
- **C* 4.13** - Build a toy model of neuron superposition that has many more hidden features to compute than output features. Ideas:
 - Have n input features and an output feature for each pair of input features, and train it to compute the max of each pair.
 - Have discrete input data, eg if it's on, take on values in `[1.0, 2.0, 3.0, 4.0, 5.0]`, and have 5 output features per input feature, with the label being `[1, 0, 0, 0, 0], [0, 1, 0, 0, 0], ...` and mean-squared error loss.
- **C* 4.14** - Build a toy model of neuron superposition that needs multiple hidden layers of ReLUs. Can computation in superposition happen across several layers? Eg

$$\max(|x|, |y|)$$

- **C-D* 4.15** - Build a toy model of *attention head* superposition/polysemanticity. Can you find a task where a model wants to be doing different things with an attention head on different inputs? How do things get represented internally? How does it deal with interference?
 - I'd recommend starting with a task involving a big ensemble of skip trigrams. The simplest kind are `A ... B -> A` (ie, if the current token is B, and token A occurred in the past, predict that A comes next).
 - **C-D* 4.16** - Build a toy model where a model needs to deal with simultaneous interference, and try to understand how it does it (or if it can do it at all!).
- Making toy models that are "counterexamples in mechanistic interpretability" - weird networks that solve tasks in ways that violate our standard intuitions for how models work ([Credit to Chris Olah for this list!](#)):
 - **C* 4.17** - A learned example of a network with a "non-linear representation". Where its activations *can* be decomposed into independently understandable features, but not in a linear way (eg via different geometric regions in activation space aka polytopes)
 - A core difficulty is that it's not clear to me how you'd distinguish between "the model has not yet computed feature X but could" and "the model *has*

computed feature X, but it is not represented as a direction". Maybe if the model can do computation within the non-linear representation, without ever needing to explicitly make it linear?

- **C* 4.18** - A network that doesn't have a discrete number of features (eg. perhaps it has an infinite regression of smaller and smaller features, or fractional features, or something else)
- **C* 4.19** - A neural network with a "non-decomposable" representation, ie where we can't break down its activations into independently understandable features
- C 4.20 - A task where networks can learn multiple different sets of features.
- Studying bottleneck superposition in real language models
 - **Tip:** To study [induction circuits](#), look at `attn-only-2l` in `TransformerLens`. To study [Indirect Object Identification](#), look at `gpt2-small`.
 - **B* 4.21** - [Induction heads](#) copy the token they attend to to the output, which involves storing which of the 50,000(!) input tokens it is in the 64 dimensional value vector. How are the token identities stored in the 64 dimensional space?
 - I'd start by using [Singular Value Decomposition](#) (and other dimensionality reduction techniques), and trying various ways to visualize how the tokens are represented in the latent space.
 - I expect part of the story is that the softmax on the logits is a very powerful non-linearity for cleaning up noise and interference.
 - **B* 4.22** - The previous token head in an induction circuit communicates the value of the previous token to the key of the induction head. As above, how is this represented?
 - Bonus: Since this is in the residual stream, what subspace does it seem to take up? Does it overlap much with anything else in the residual stream? Can you find any examples of interference?
 - **B* 4.23** - The [Indirect Object Identification circuit](#) communicates names or positions between the pairs of composing heads. How is this represented in the residual stream? How many dimensions does it take up?
 - **B* 4.24** - In models like GPT-2 with absolute positional embeddings, knowing this positional information is extremely important, so the ReLU output model predicts that these should be given dedicated dimensions. Does this happen? Can you find any other components that write to these dimensions?
 - Note that the positional embeddings as a whole tend to only take up a few dimensions (6-20 of GPT-2 Small's 768 residual stream dimensions). You

can find this with a Singular Value Decomposition. I would focus solely on the important dimensions.

- Note also that the *first* positional embedding is often weird, and I would ignore it.
- **C-D* 4.25** - Can you find any examples of the geometric superposition configurations from the ReLU output model in the residual stream of a language model?
 - I think antipodal pairs are the most likely to occur.
 - I recommend studying the embedding or unembedding and looking for highly anti-correlated features.
 - One example is the difference between similar tokens - the difference between the Tuesday embedding and the Wednesday embedding only really matters if you're confident that the next token is a day of the week. So there should be no important interference with eg a "this is Python code" feature, which is just in a totally different context.
- **C* 4.26** - Can you find any examples of [locally almost-orthogonal bases](#)? That is, where correlated features each get their own direction, but can interfere significantly with un/anti-correlated features.
- **C* 4.27** - I speculate that an easy way to do bottleneck superposition with language data is to have "genre" directions which detect the type of text (newspaper article, science fiction novel, wikipedia article, Python code, etc), and then to represent features specific to each genre in the *same* subspace. Because language tends to fall sharply into one type of text (or none of them), the model can use the same genre feature to distinguish many other sub-features. Can you find any evidence for this hypothesis?
- **D* 4.28** - Can you find any examples of a model learning to deal with simultaneous interference? Ie having a dimension correspond to multiple features and being able to deal sensibly with both being present?
- Studying neuron superposition in real models:
 - **B* 4.29** - Look at a polysemantic neuron in a one layer language model. Can you figure out how the model disambiguates which feature it is?
 - I'd start by looking for polysemantic neurons in [neuroscope](#)
 - 1L is an easy place to start because a neuron can *only* impact the output logits, so there's not that much room for complexity
 - **C* 4.30** - Do this on a two layer language model.

- **B* 4.31** - Take one of the features that's part of a polysemantic neuron in a 1L language model and try to identify every neuron that represents that feature (such that if you eg use activation patching on just those neurons, the model completely cannot detect the feature). Is this sparse (only done by a few neurons) or diffuse (across many neurons)?
 - **C* 4.32** - Try to fully reverse engineer that feature! See if you can understand how it's being computed, and how the model deals with alternating or simultaneous interference.
- **C* 4.33** - Can you use superposition to create an adversarial example for a model?
 - I'd start by looking for polysemantic neurons in [neuroscope](#) and trying to think of a prompt which contains multiple features that strongly activate that neuron.
 - Any of the other ideas in this section could motivate a similar problem, eg finding names that interfere a lot for the IOI task.
- C 4.34 - Can you find any examples of the asymmetric superposition motif in the MLP layer of a one or two layer language model?
- Trying to find the direction corresponding a specific feature:
 - *This is a widely studied subfield of interpretability (including non-mechanistic!) called [probing](#), see a [literature review](#). In brief, it looks like taking a dataset of positive and negative examples of a feature, looking at model activations on both, and finding a direction that predicts the presence of the feature.*
 - **C-D* 4.35** - Pick a simple feature of language (eg "is a number" or "is in base64") and train a linear probe to detect that in the MLP activations of a one layer language model (there's a range of possible methods! I'm not sure what's suitable). Can you detect the feature? And if so, how sparse is this probe? Try to explore and figure out how confident you are that the probe has actually found how the feature is represented in the model?
 - **C-D* 4.36** - Look for features in [neuroscope](#) that seem to be represented by various neurons in a 1L or 2L language model. Train a probe to detect some of them, and compare the performance of these probes to just taking that neuron. Explore and try to figure out how much you think the probe has found the true feature
- Comparing SoLU and GELU
 - *The [SoLU paper](#) introduces the SoLU activation and claims that it leads to more interpretable and less polysemantic neurons than GELU*

- **A* 4.37** - How do my SoLU and GELU models compare in [neuroscope](#) under the polysemanticity metric used in the SoLU paper? (what fraction of neurons seem monosemantic when looking at the top 10 activating dataset examples for 1 minute)
 - **B* 4.38** - The SoLU metrics for polysemanticity are somewhat limited. Can you find any better metrics? Can you be more reliable, or more scalable?
- **B-C* 4.39** - The paper speculates that the LayerNorm after the SoLU activations lets the model "smuggle through" superposition, by smearing features across many dimensions, having the output be very small, and letting the LayerNorm scale it up. Can you find any evidence of this in `solu-1l`?
 - I would start by checking how much the LayerNorm scale factor varies between tokens/inputs.
 - I would also look for tokens where the neuron has (comparatively) low activation pre-LayerNorm but high direct logit attribution
 - I would also look for tokens where the direct logit attribution of the MLP layer is high, but no single neuron is high.
- B 4.40 - I have pairs of 1L-4L toy models trained with SoLU or with GELU activations, but otherwise the same weight initialisation and data shuffle. How do they differ? How similar are the neurons between the two?
 - (My guess is the answer is "they're just totally different", but I'm curious!)
- C 4.41 - How does GELU vs ReLU compare re polysemanticity? That is, train a small model with ReLU vs GELU and try to replicate the SoLU analysis there
- Getting rid of superposition (at the cost of performance) - can we achieve this at all?
 - **C* 4.42** - If you train a 1L or 2L language model with $d_{\text{mlp}} = 100 * d_{\text{model}}$, what happens? Does superposition go away? In theory it should have more than enough neurons for one neuron per feature.
 - Alternately, it may learn incredibly niche and sparse features and just pack these into the original neurons
 - Warning: Training a language model, even a toy one, can be a *lot* of effort!
 - C 4.43 - The original T5 XXL may be useful to study here, $d_{\text{model}}=1024$, $d_{\text{mlp}}=65536$
 - But it's 11B parameters, so will also be really hard for other reasons, and is encoder-decoder so it's not supported by TransformerLens. Expect major infrastructure pain!

- **D* 4.44** - Can you take a trained model, freeze all weights apart from a single MLP layer, then make the MLP layer 10x the width, copy each neuron 10 times, add some noise and fine-tune? Does this get rid of superposition? Does it add in new features?
- **C-D* 4.45** - There's a long list of open questions at the end of [Toy Models](#). Pick one and try to make progress on it!

Previous:

200 COP in MI: Interpreting Algorithmic Problems

No comments 33 karma

Next:

200 COP in MI: Analysing Training Dynamics

No comments 16 karma

[Log in to save where you left off](#)

Mentioned in

- 41 200 Concrete Open Problems in Mechanistic Interpretability: Introduction
- 14 Mechanistic Interpretability Quickstart Guide
- 11 200 COP in MI: Studying Learned Features in Language Models

1 comment, sorted by [top scoring](#)[-] **LawrenceC** 2y ▼ 2 ▲ ✕ 0 ✓

I strongly upvoted this post because of the "Tips" section, which is something I've come around on only in the last ~2.5 months.

[Moderation Log](#)

More from Neel Nanda

19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0
32	Models May Behave Worse When Eval Aware...	Senthooran Rajamanoharan, Neel ...	2d	0
65	models have some pretty funny attractor sta...	aryaj, Senthooran Rajamanoharan, ...	3mo	0

View more

Curated and popular this week

39	Sympathy for both sides of the egregious misalignment debate	Steven Byrnes	7h	5
26	A Mike’s-Eye View of ARC’s Research	Mikewins	3d	0
19	Building and evaluating model diffing agents	bilalchughtai, Josh Engels, Neel Nanda	6h	0